

Table 2-8: Halving/Doubling Table, Part 8

Halving	Doubling
$(2^5 + 2^3 + 2^2) \times 2^1 + 2^0 =$ $2^6 + 2^4 + 2^3 + 2^0$	18×2^0
$(2^4 + 2^2 + 2^1) \times 2^1 =$ $2^5 + 2^3 + 2^2$	18×2^1
$(2^3 + 2^1 + 2^0) \times 2^1 = 2^4 + 2^2 + 2^1$	18×2^2
$(2^2 + 2^0) \times 2^1 + 2^0 = 2^3 + 2^1 + 2^0$	18×2^3
$2^1 \times 2^1 + 2^0 = 2^2 + 2^0$	18×2^4
$2^0 \times 2^1 = 2^1$	18×2^5
2^0	18×2^6

If you number the rows of the halving column starting with the top row as row 0, then 1, 2, and all the way to the bottom row as row 6, you can see that the rows with odd values in the halving column are rows 0, 3, 4, and 6. Now notice the crucial pattern: those row numbers are exactly the exponents in the expression for 89 that we found: $2^6 + 2^4 + 2^3 + 2^0$. This is not a coincidence; the way we constructed the halving column means that the odd entries will always have row numbers that are the exponents in a sum of powers of 2 equaling our original number. When we take a sum of the doubling entries with those indices, we're summing up 18 multiplied by powers of 2 that sum to exactly 89, so we'll get 89×18 as our result.

The reason this works is that really, RPM is an algorithm within an algorithm. The halving column itself is an implementation of an algorithm that finds the sum of powers of 2 that equals the number at the top of the column. This sum of powers of 2 is also called the *binary expansion* of 89. Binary is an alternative way to write numbers using only 0s and 1s, and it has become extremely important in recent decades because computers store information in binary. We can write 89 in binary as 1011001, with 1s in the zeroth, third, fourth, and sixth places (counting from the right), the same as the odd rows of the halving column, and also the same as the exponents in our equation. We can interpret the 1s and 0s in a binary representation as coefficients in a sum of powers of 2. For example, if we write 100, we interpret it in binary as

$$1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$$